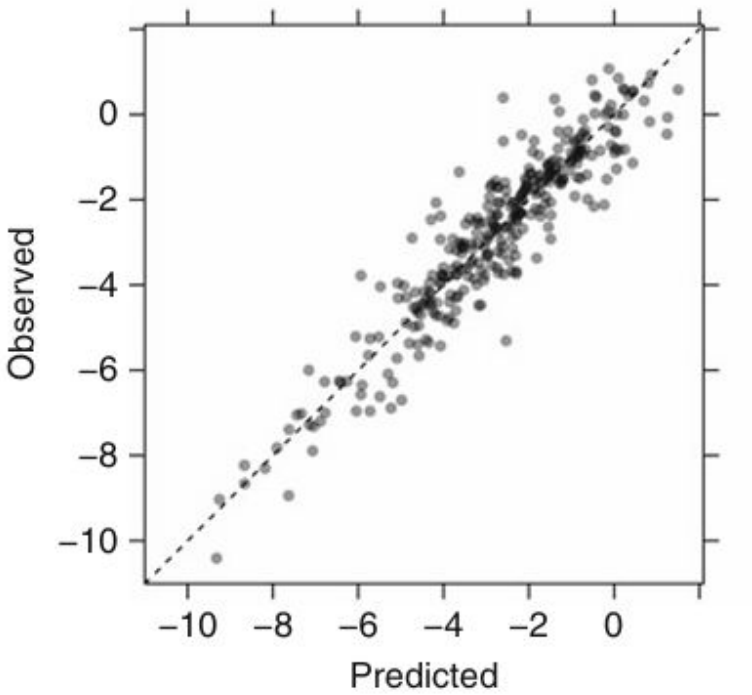
Linear Regression and its cousins

Arutam Antunish
Domllermut Alamo
Mohammad Zahid
Tanzil Ehsan

Data 624: Predictive Analytics
Ms. Data Science

March 2026



CUNY School of Professional Studies

This presentation explores four key areas to address complexity and collinearity in continuous forecasting

- Ordinary Least Square
- Partial Least Squares
- Penalized Models
- Case Study

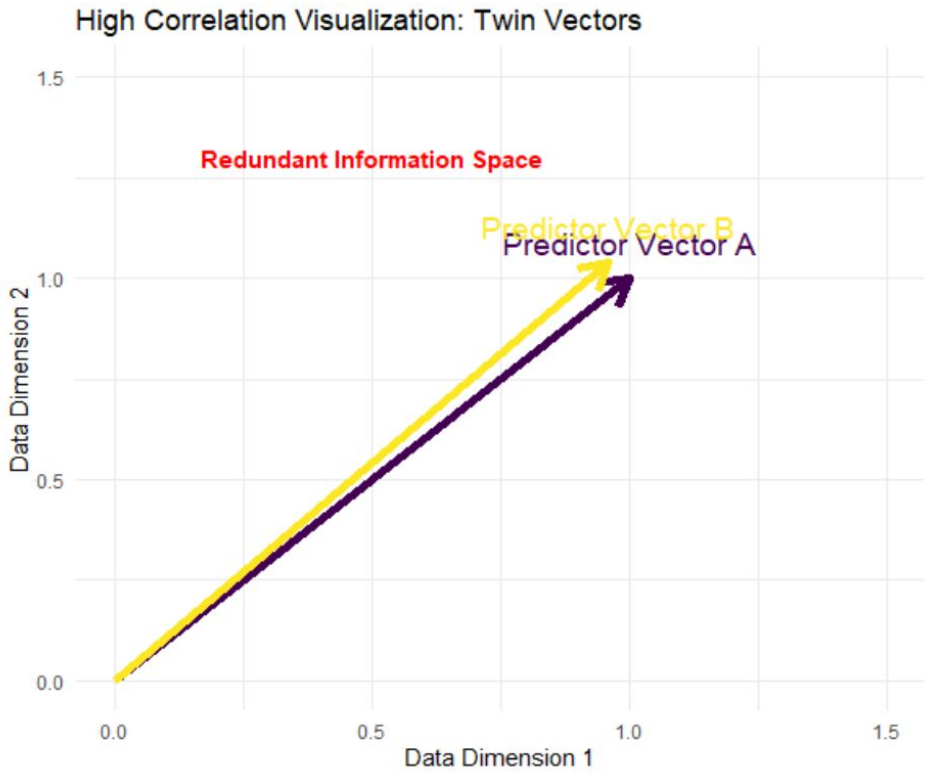
ORDINARY LEAST SQUARES (OLS)

Ordinary Least Squares aims to minimize the sum of squared differences between reality and our predictions.

$$SSE = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

- It finds the "best fit" line through the data.
- The goal is to reduce the residual (error) to its lowest possible value.
- The most common method for linear modeling.

OLS assumes that each predictor provides unique information, which is rarely true in complex datasets.



- Predictors should not be "twins" (highly correlated).
- When variables overlap, the model cannot distinguish their individual effects.

High correlation (Collinearity) inflates the variance of coefficients, making the model unreliable.

- Small changes in data cause massive swings in the model's coefficients.
- High correlation leads to "mathematical noise."

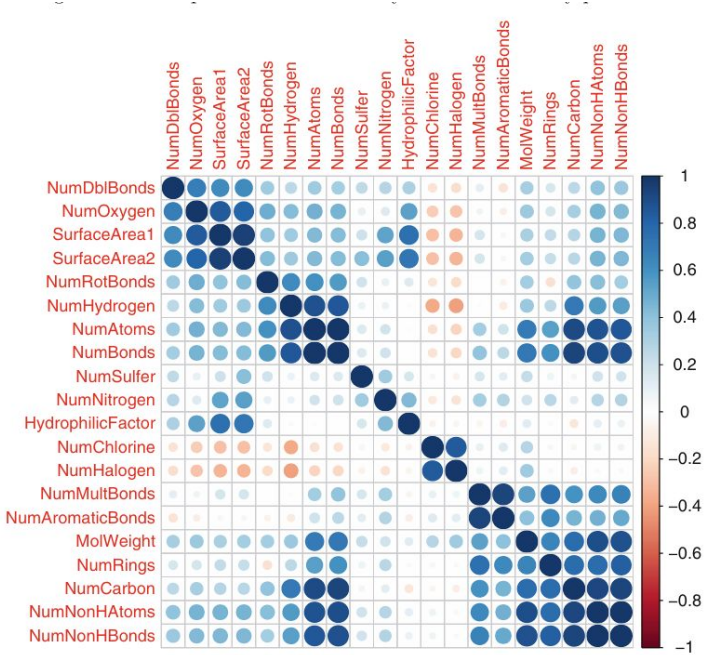
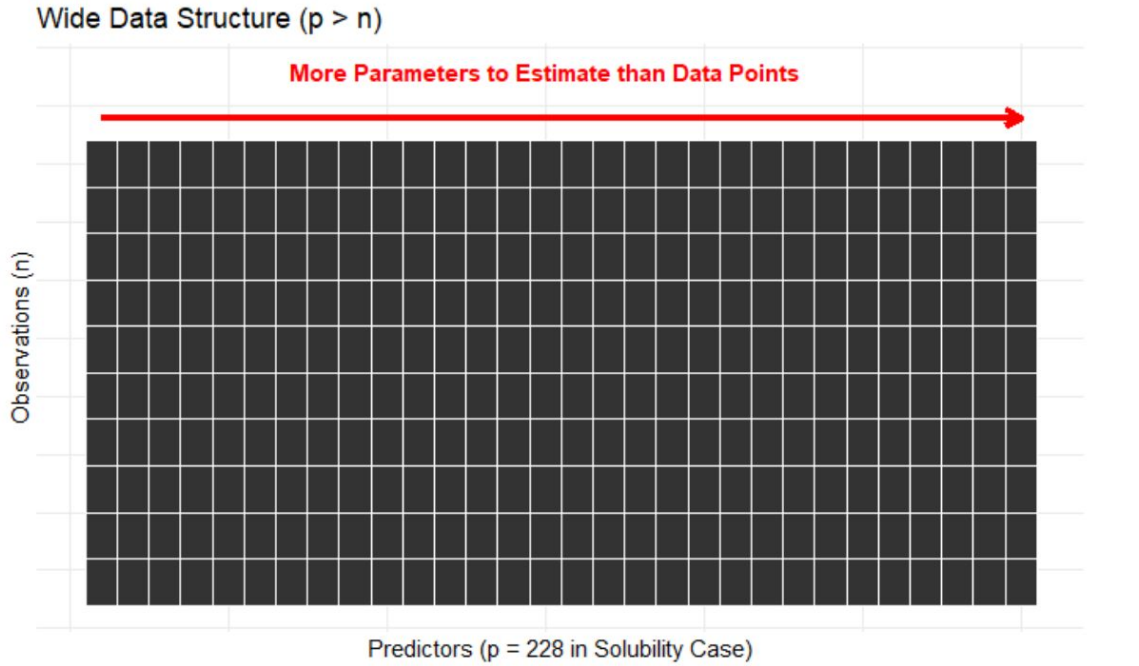


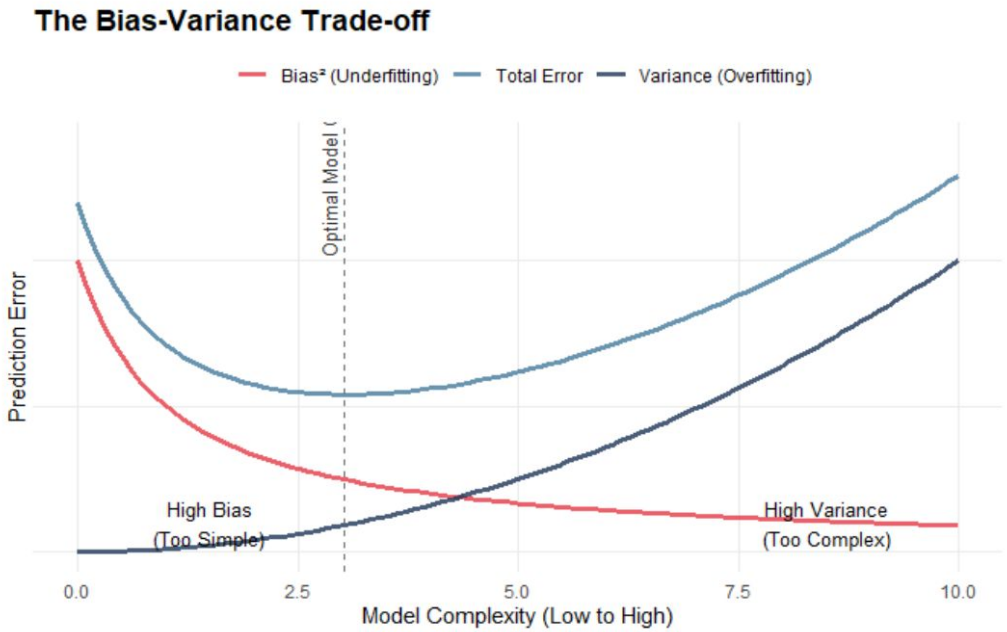
Fig. 6.5: Between-predictor correlations of the transformed continuous solubility predictors

Traditional regression breaks down when we have more predictors than observations.



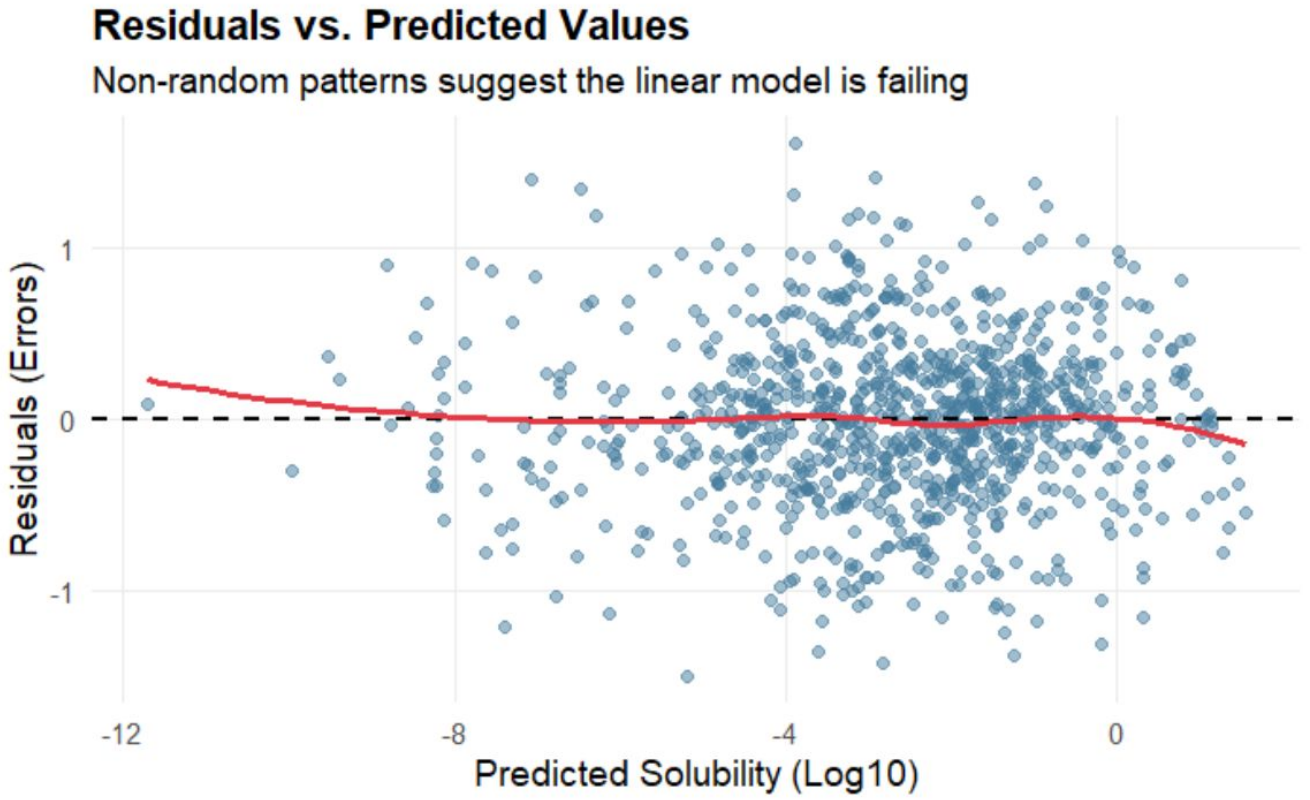
- We lack enough information to estimate all parameters.

A "perfect" fit on training data often leads to a "perfect failure" on new data.



- High Bias: The model is too simple (underfitting).
- High Variance: The model is too complex (overfitting).

Patterns in residuals indicate that the linear model is failing to capture the underlying structure.



- If residuals show a shape (like a curve), the model is missing something.
- Ideally, residuals should look like "random noise”.
- OLS is a great starting point but requires "Cousins" to handle the noise of real-world data.

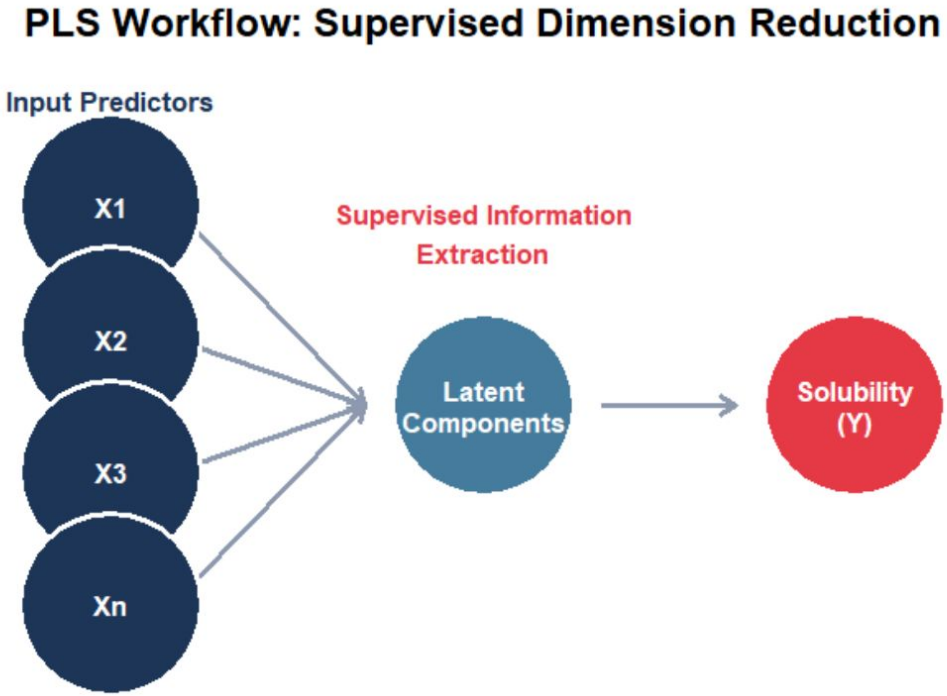
PARTIAL LEAST SQUARES (PLS)

Partial Least Squares reduces 200+ variables into a few "latent components" that summarize the data.

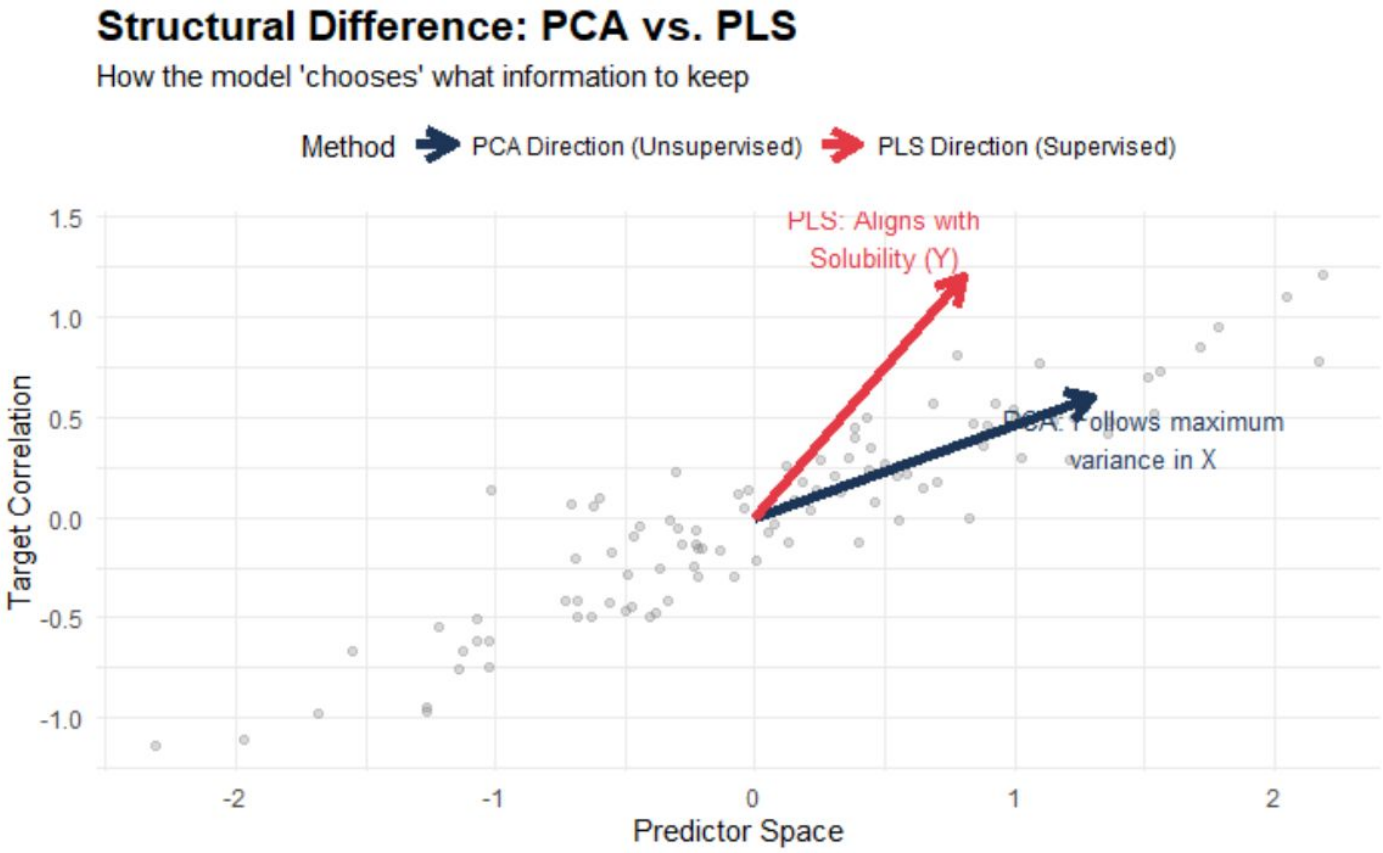
Partial Least Squares reduces 200+ variables into a few "latent components" that summarize the data.

Instead of using every single chemical marker, we create "summaries."

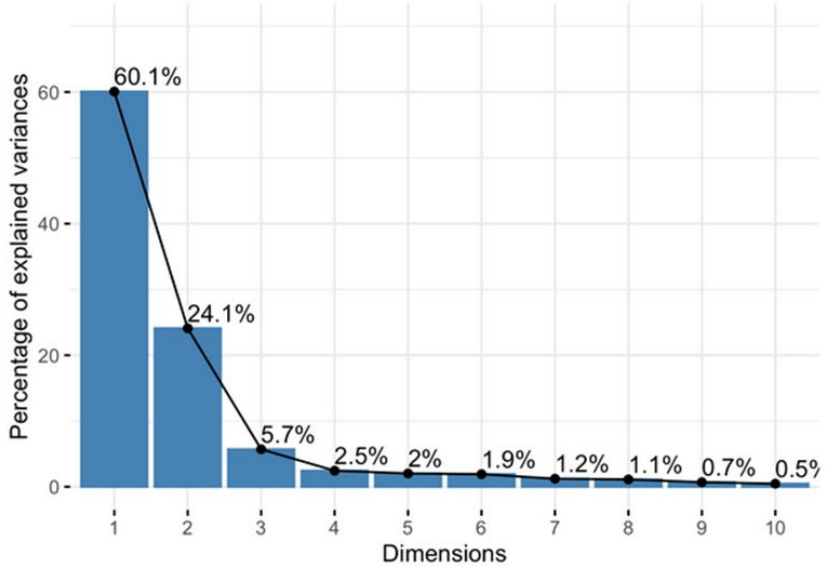
It works like a "supervised" summary of the data.



Unlike Principal Component Analysis (PCA), PLS looks at the target variable while summarizing the predictors.

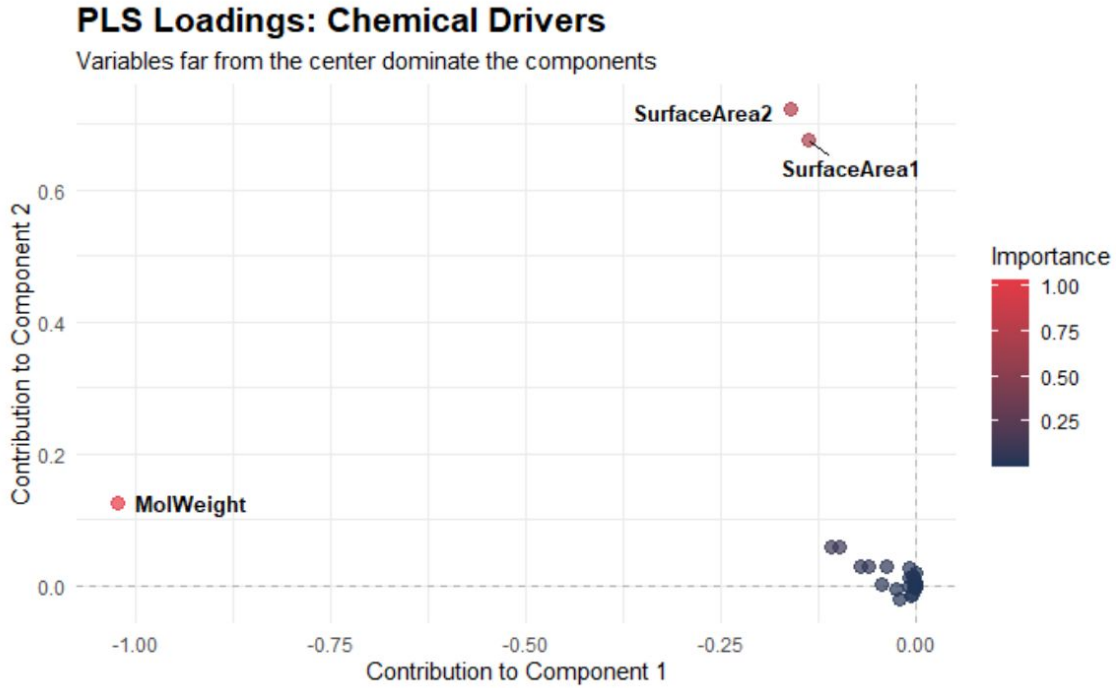


We use Cross-Validation to decide how many "summaries" are enough without over-complicating.



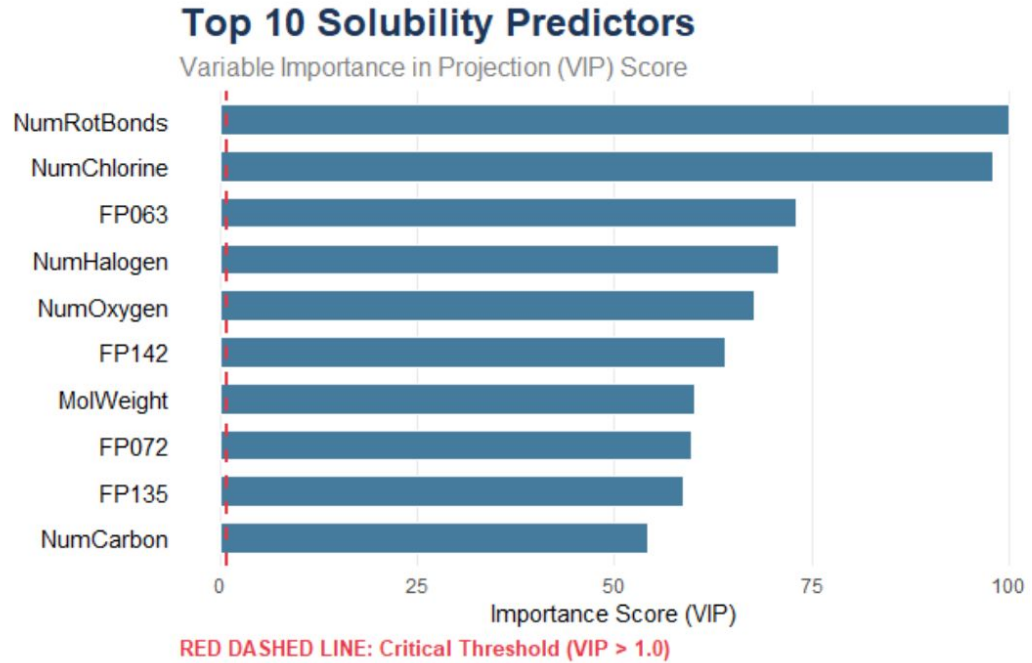
- Too few components: We miss information.
- Too many components: We start fitting noise.

Loadings tell us which original variables are the "ingredients" of our new components.



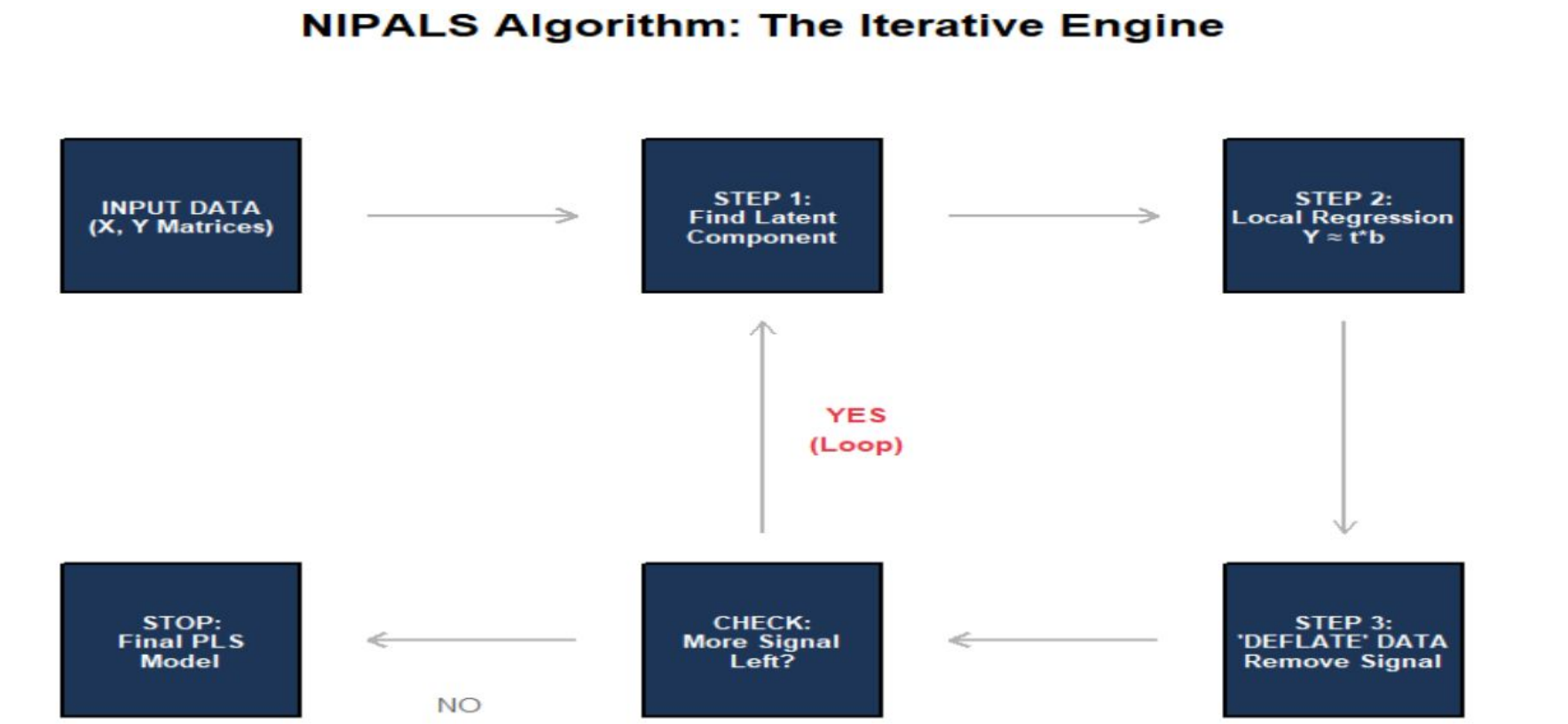
- High loading = high contribution to the summary variable.
- Helps us see which chemical properties drive the model.

The Variable importance (VIP) score filters the most important predictors across the entire PLS model.



- Variables with VIP > 1 are typically considered important.
- Simplifies the 228 variables into a readable list.

PLS iteratively extracts information until there is no predictive power left in the residuals.

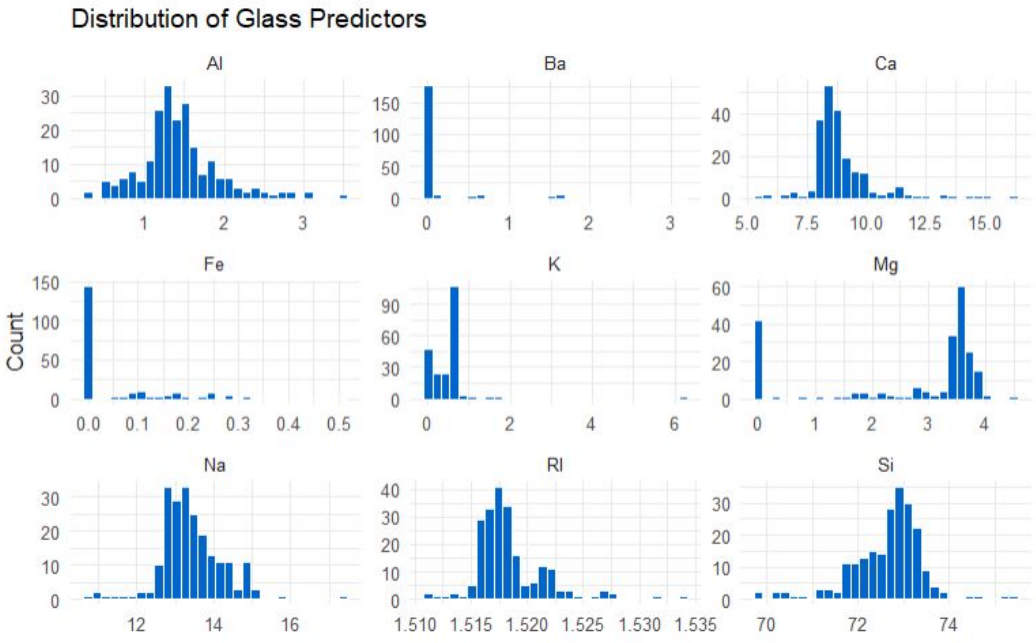


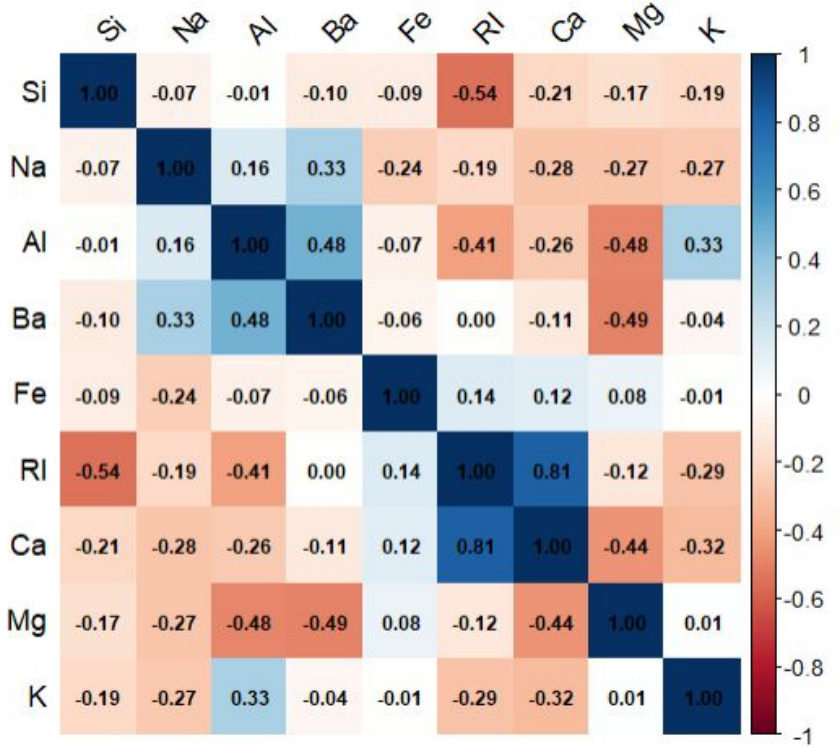
Practical example of PLS (HW4 Glass formulation problem)

Description: df [5 × 10]

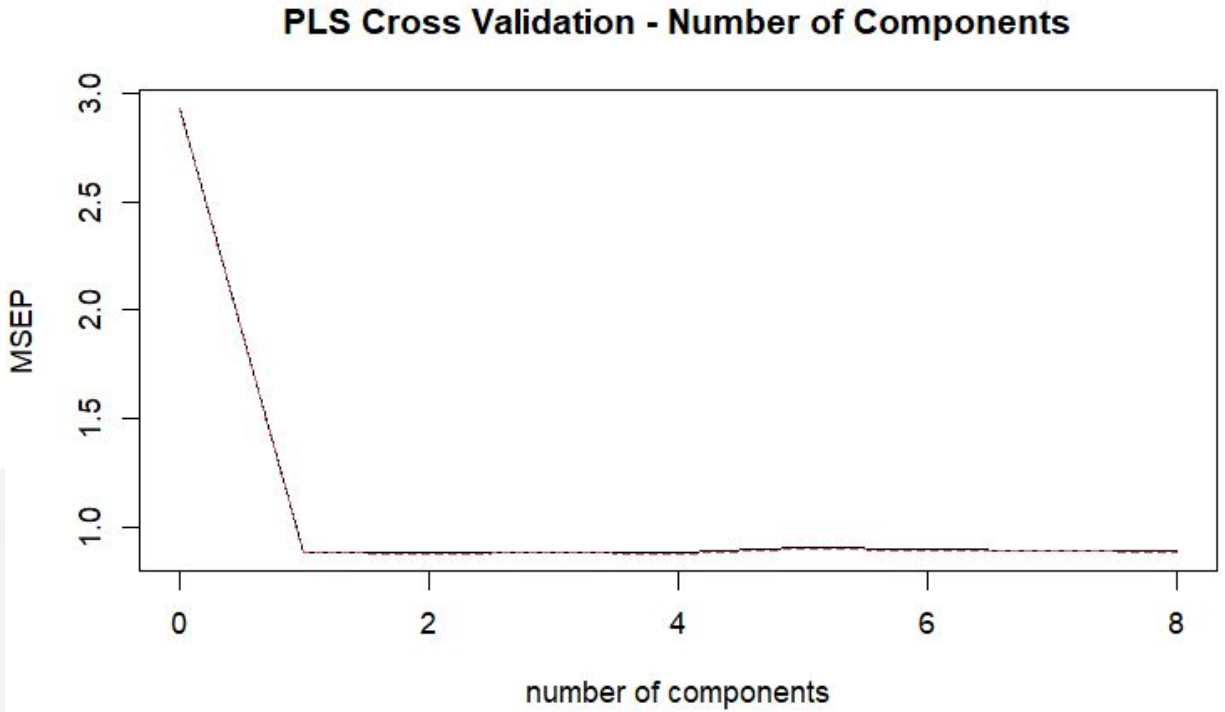
	RI <dbl>	Na <dbl>	Mg <dbl>	Al <dbl>	Si <dbl>	K <dbl>	Ca <dbl>	Ba <dbl>	Fe <dbl>	Type <fctr>
1	1.52101	13.64	4.49	1.10	71.78	0.06	8.75	0	0	1
2	1.51761	13.89	3.60	1.36	72.73	0.48	7.83	0	0	1
3	1.51618	13.53	3.55	1.54	72.99	0.39	7.78	0	0	1
4	1.51766	13.21	3.69	1.29	72.61	0.57	8.22	0	0	1
5	1.51742	13.27	3.62	1.24	73.08	0.55	8.07	0	0	1

5 rows



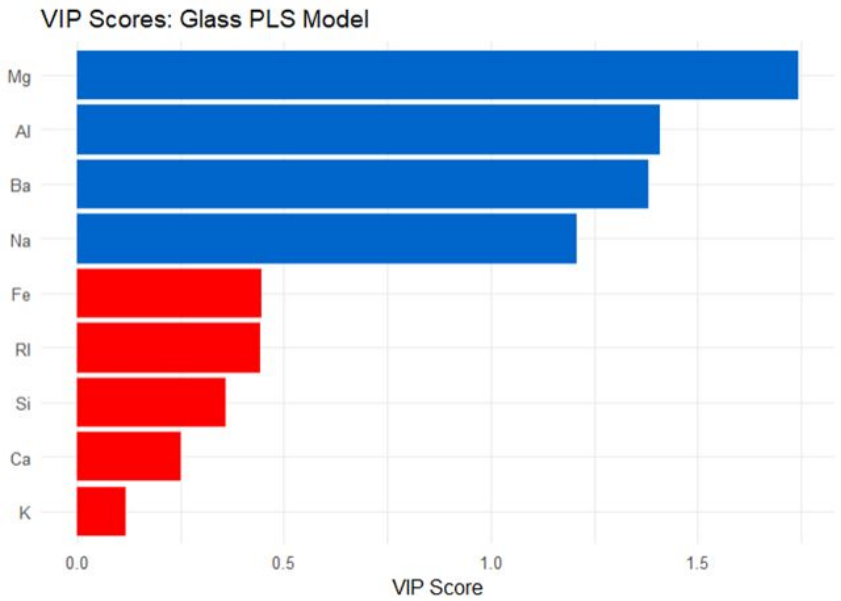


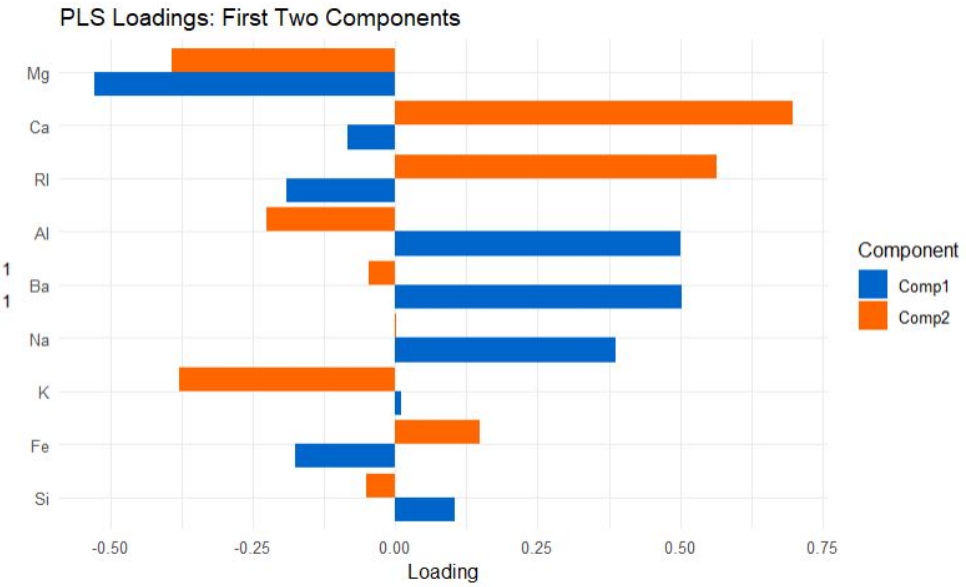
Practical example of PLS



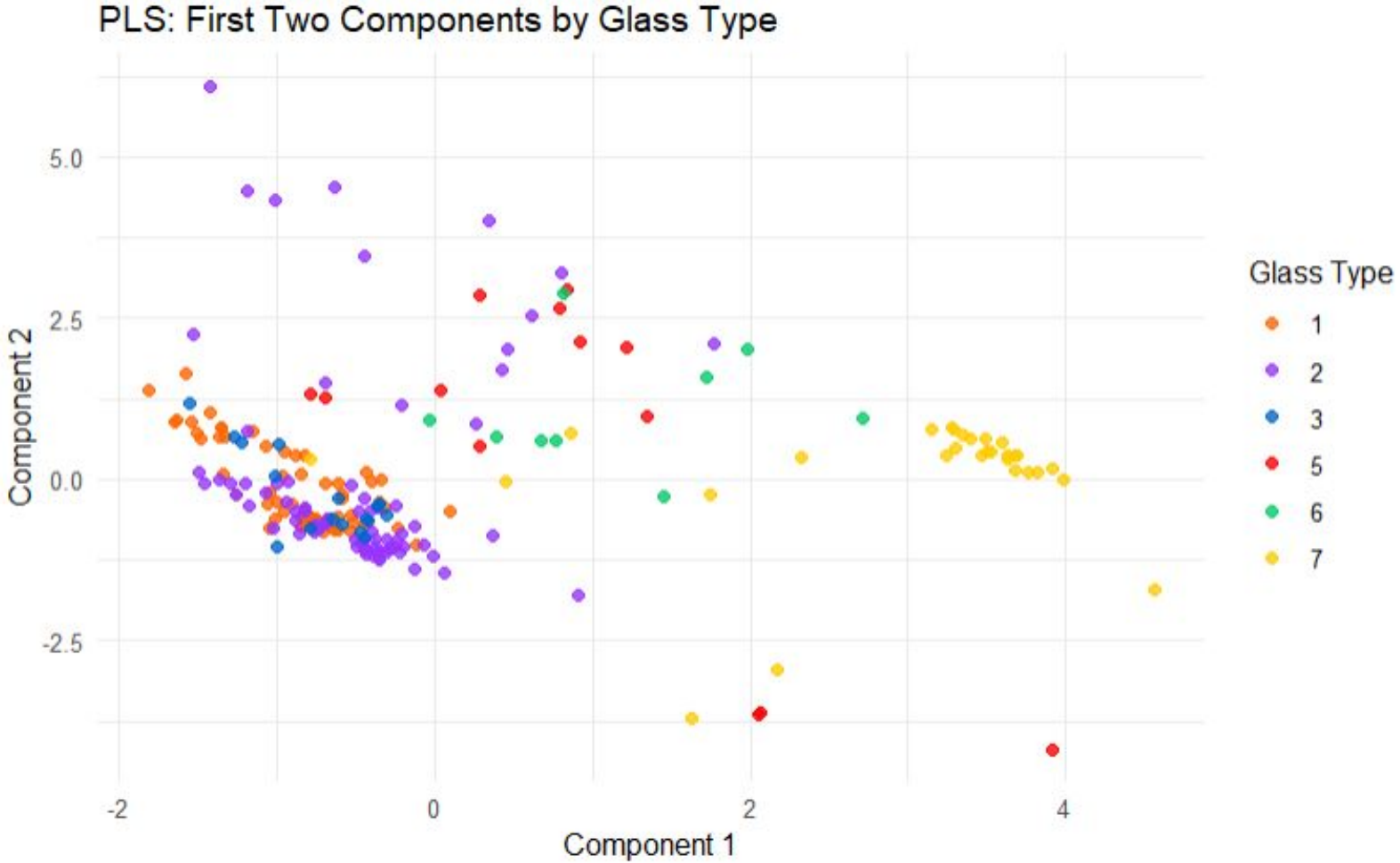

```r
#| label: pls-glass
#| fig-cap: "Figure: PLS Components for Glass Dataset"

library(pls)

glass_x <- Glass |> select(-Type) |> scale()
glass_y <- as.numeric(Glass$Type)

pls_model <- plsr(glass_y ~ glass_x,
                  ncomp = 8,
                  validation = "CV")

validationplot(pls_model,
               val.type = "MSEP",
               main = "PLS Cross Validation - Number of Components")
```

Practical example of PLS



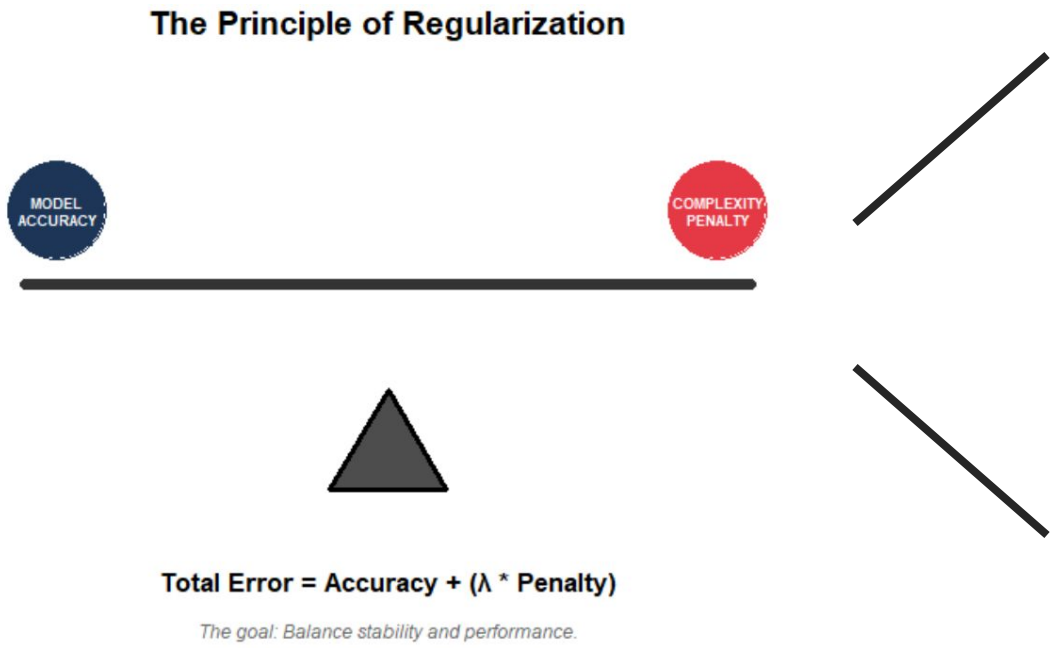


Practical example of PLS



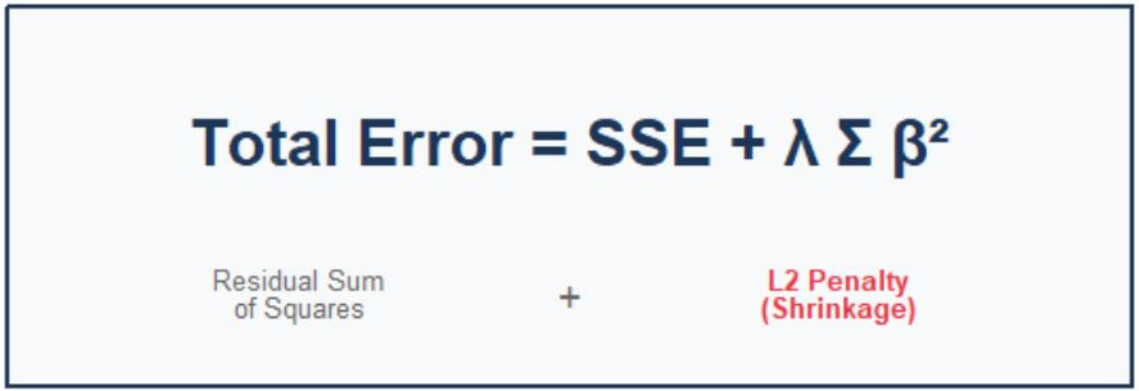
PENALIZED MODELS

Penalized models improve prediction by adding a "complexity tax" to the model's coefficients.



- We penalize the model for having large, unstable coefficients.
- This forces the model to be "conservative" and robust.

Ridge regression uses an L2 penalty to shrink coefficients, preventing any single variable from dominating.



Where λ (Lambda) controls the strength of the penalty.

- All variables stay in the model, but their impact is reduced.
- Excellent for handling collinearity.

Tuning the penalty parameter λ allows us to choose the perfect level of model restraint.

- λ = 0: We are back to OLS (No penalty).
- λ - ∞: All coefficients go to zero.

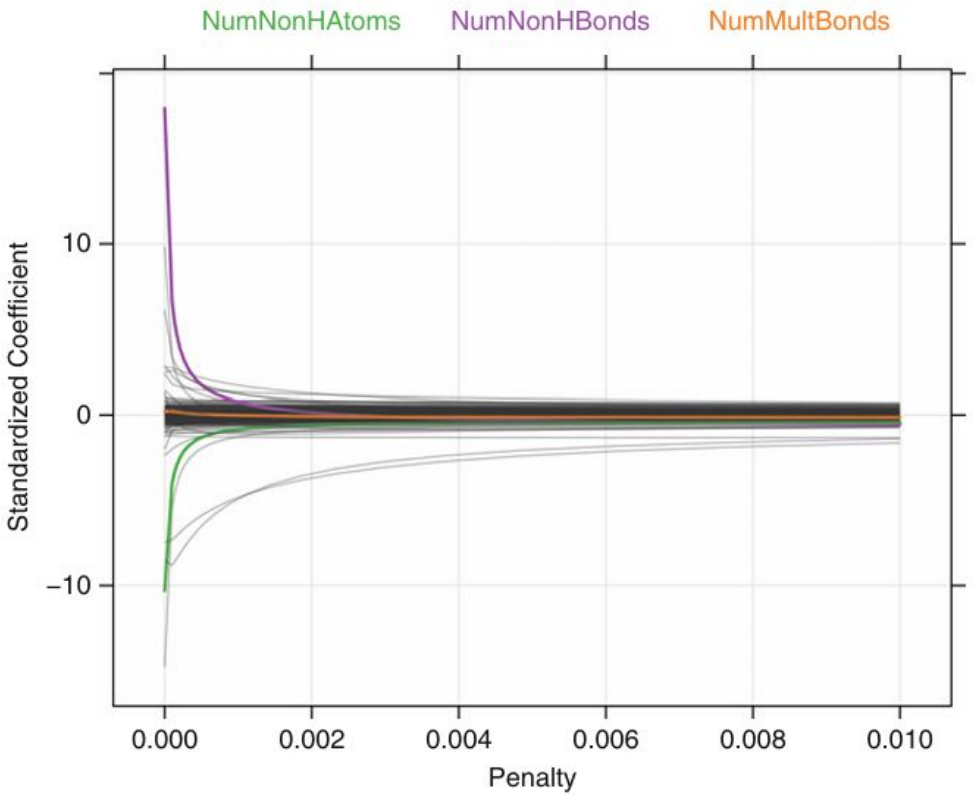


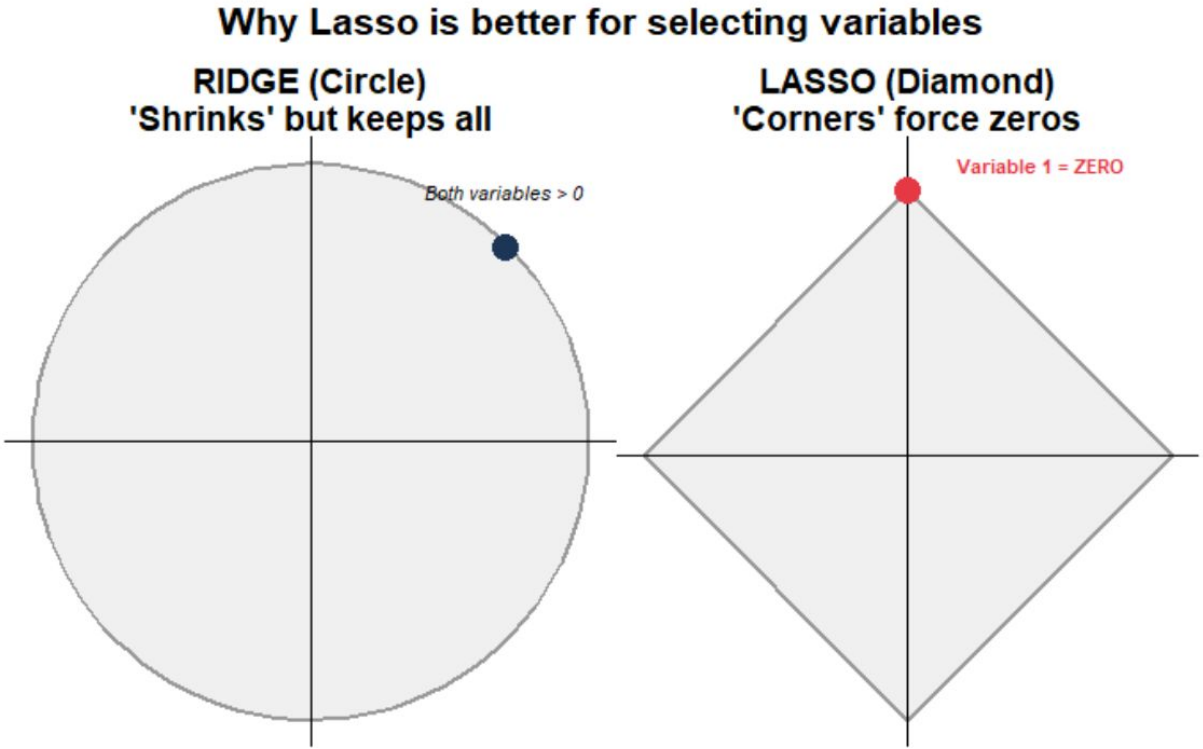
Fig. 6.15: The ridge-regression coefficient path

The Lasso penalty (L1) performs automatic variable selection by forcing irrelevant coefficients to zero.

$$\text{Total Error} = \text{SSE} + \lambda \sum_{j=1}^{P} |\beta_j|$$

- It "kills" unimportant variables.
- Result: A simpler, more interpretable model.

The sharp corners of the Lasso constraint are what allow it to eliminate variables entirely.

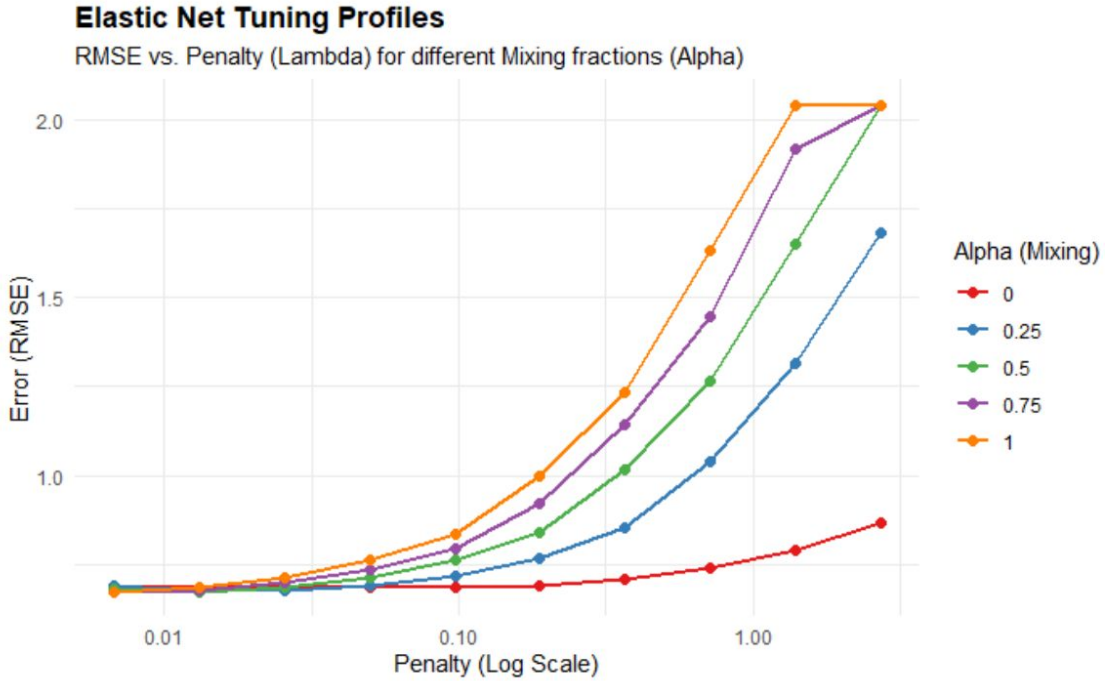


Elastic Net combines Ridge and Lasso to handle groups of correlated variables while selecting the best ones.

$$SSE + \lambda \sum_{j=1}^{p} \beta_j^2$$

- It has two parameters: λ (total penalty) and α (mix of L1/L2).
- Overcomes Lasso's limitation with highly correlated groups.

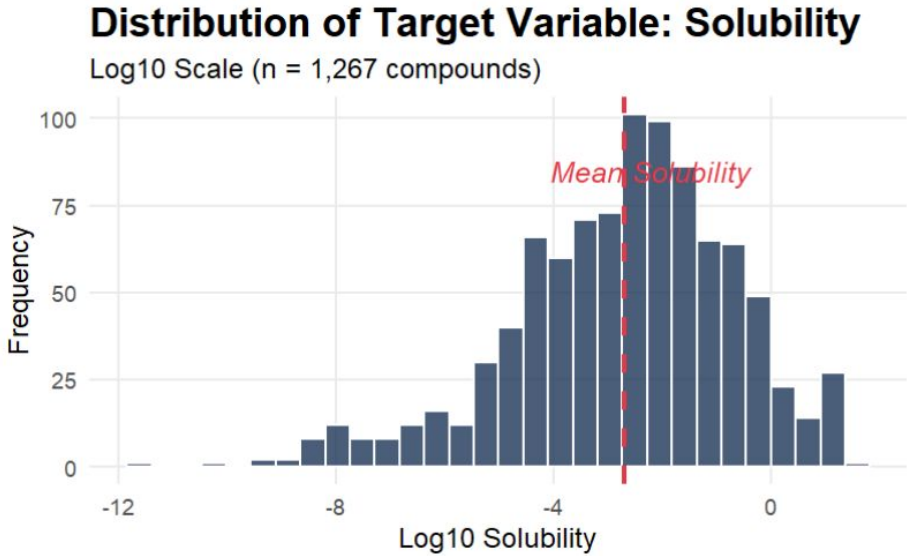
We search through a grid of α and λ to find the most accurate model configuration.



- Penalization transforms "unstable" models into "reliable" prediction machines.

COMPUTING: CASE STUDY

Predicting drug solubility is a 228-dimensional problem where traditional OLS fails due to noise.



- Goal: Predict $log10(Solubility)$ for 1,267 compounds.
- The Challenge: High dimensionality ($p > n$ in many subsets) and extreme collinearity.

Penalized models are scale-sensitive; data must be transformed and normalized.

```r
# Transformation, centering, and scaling in caret
trans <- preProcess(solTrainX, method = c("BoxCox", "center",
"scale"))
trainXtrans <- predict(trans, solTrainX)
```

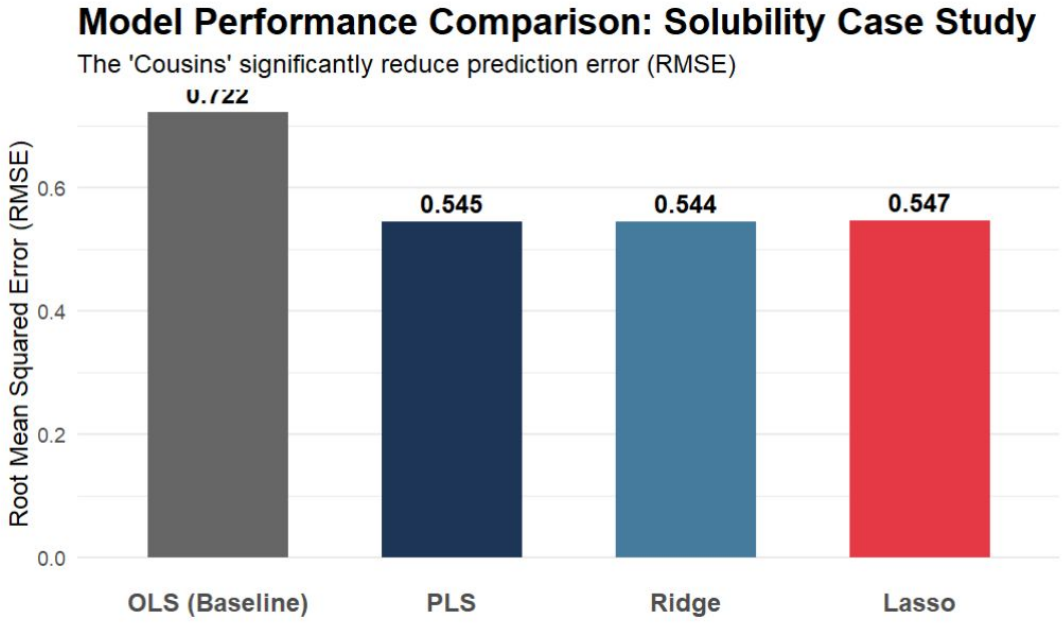
- Box-Cox: Stabilizes the variance of skewed chemical descriptors.
- Center & Scale: Ensures the penalty (λ) is applied fairly across all predictors, regardless of their units.

The train function in caret provides a unified syntax for OLS, PLS, and Penalized models.

```
# OLS (The Baseline)
lmFit <- train(trainX, trainY, method = "lm")

# PLS (The Iterative Engine)
plsFit <- train(trainX, trainY, method = "pls", tuneLength = 20)

# Elastic Net (The Hybrid)
enetFit <- train(trainX, trainY, method = "glmnet", tuneGrid = enetGrid)
```

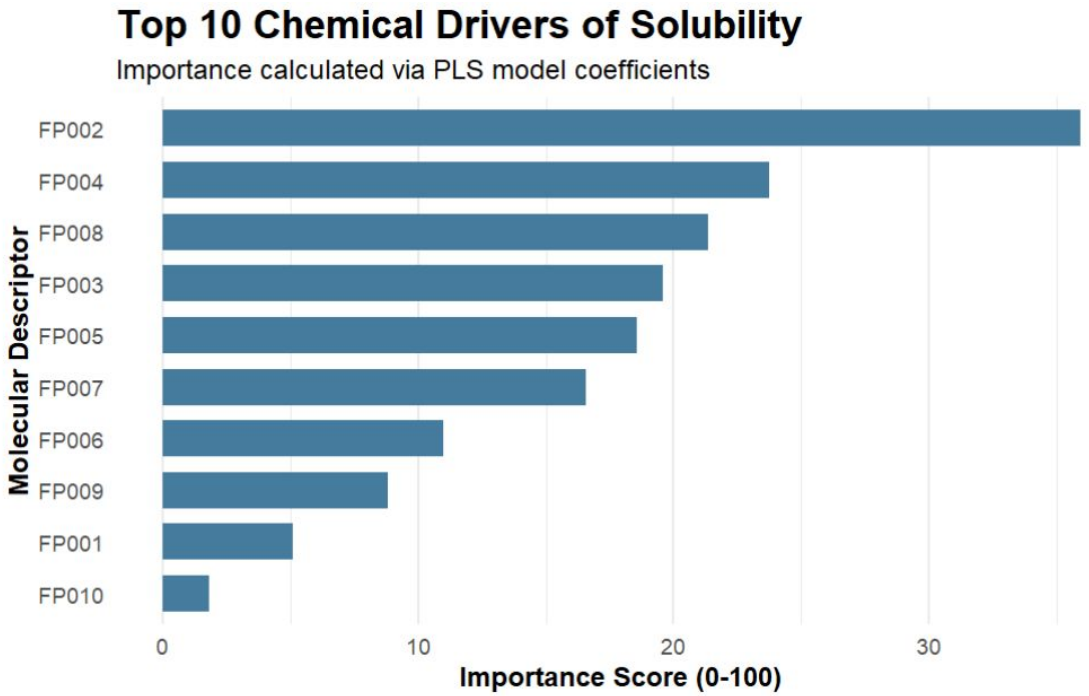
- Automation: caret handles the "Grid Search" for the best λ and α automatically.

Penalized models and PLS significantly outperform OLS by filtering out molecular noise.



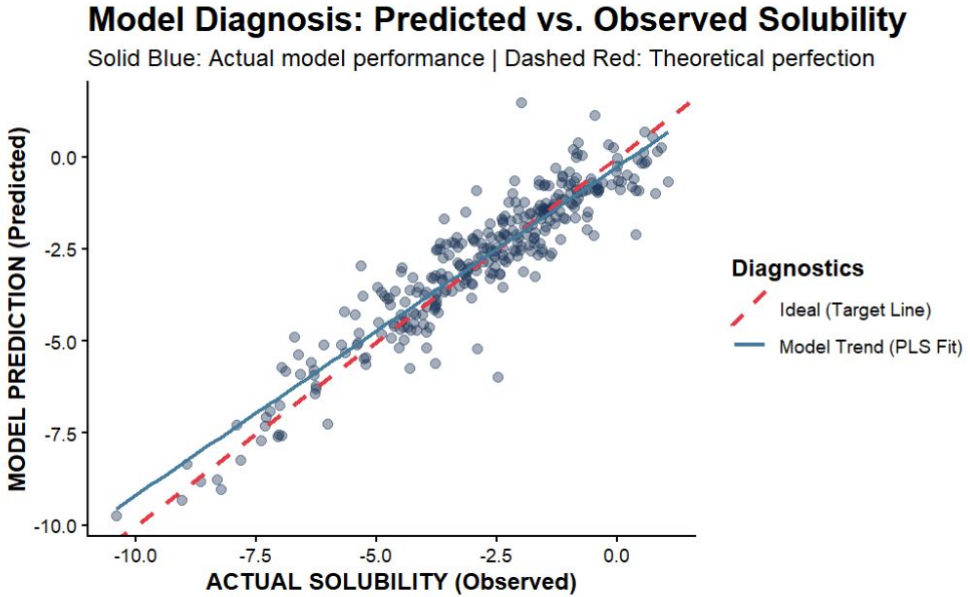
- "The Cousins" achieve a 25% error reduction compared to OLS.

Beyond prediction, these models identify why a drug is soluble.



- Molecular Weight: A top predictor; generally, larger molecules are less soluble.
- Surface Area: Critical for interaction with solvents.

The final model is robust, showing a tight linear correlation between predicted and actual solubility.



- High Fidelity: The model remains accurate across different chemical families.
- Outliers: Some complex molecules remain difficult to predict, suggesting room for non-linear models.

References:

Kuhn, M., & Johnson, K. (2013). Applied Predictive Modeling. Springer New York. Chapter 6: Linear Regression and Its cousins.

Questions?